

Report 107: Z3 SMT Solver, LIR Codegen & Frill Frontend

Report 107: Z3 SMT Solver, LIR Codegen Fixes & Frill Frontend

Date: 2026-03-22 **Branch:** feat/z3-optimal-codegen **Commits:** 24 **LOC:** +3993 lines across 23 files

Executive Summary

Single-session sprint: integrated Z3 SMT solver for provably optimal register allocation, fixed 10+ codegen bugs, added equality saturation framework, created Frill ML-inspired frontend, and improved E2E corpus from 86% to 92.6%.

E2E corpus: 68/79 (86%) → 75/81 (92.6%) — 8 frontends

1. Z3 SMT-Based Optimal Register Allocation

Z3 theorem prover integrated as subprocess for register allocation. Encodes regalloc as SMT-LIB2 optimization and finds provably optimal assignment.

```
mz program.nanz -b z80 --z3      # Z3 optimal (100ms/function)
mz program.nanz -b z80           # WFC greedy (1ms, default)
```

Key files

- z3solve.go (350 LOC) — SMT encoding, subprocess, model parsing
- z3_joint.go (190 LOC) — Joint isel+regalloc via e-graph + Z3
- z3validate.go (80 LOC) — Post-WFC consistency check + Z3 repair
- saturate.go (165 LOC) — Equality saturation with Z80 rewrite rules

PBQP hints

Z3 accepts PBQP allocations as soft preferences (+10 penalty for deviation), ensuring bootstrap compatibility for assert tests.

2. Codegen Bug Fixes (10 bugs fixed)

Bug	Before	After	Impact
addr_of symbol	LD HL, 0 (store to ROM!)	LD HL, p_name	Critical
fixInvalidZ80Template	double → 6 inst store	ADD HL, HL (1 inst!)	Critical
INC vs double	add(x,x) → INC (+1)	ADD HL, HL (double)	High
Z80 register collision	LD IY, I (I register!)	LD IY, v_I	High
16-bit cond_ret	LD A, DE (invalid!)	EX DE, HL	High
16-bit CMP pattern	CP B (8-bit truncated)	SBC HL, DE; ADD HL, DE	High
Assert bootstrap	PBQP/LIR register mismatch	Z3 soft hints	Medium
\$ in symbols	CALL fs\$fat12\$...	CALL fs_fat12_...	Medium
String pool missing	undefined labels	EmitStringPool in both paths	Medium
Tail-call safety	JP f losing return value	Only when safe	Medium

3. ASM Quality — Perfect Leaf Functions

```
; double16(x) = x + x          PERFECT (was 6 instructions!)
double16:  ADD HL, HL / RET

; add(a, b) = a + b            PERFECT
add:      ADD A, C / RET

; main = add(3, 4)             PERFECT (constant folded)
main:     LD A, 7 / RET

; main = 5 |> double |> inc     PERFECT (pipe + const fold)
main:     LD A, 11 / RET

; max(3, 7) — 16-bit compare    CORRECT
max:      OR A / SBC HL, DE / ADD HL, DE / JR NC, ... / EX DE,
HL / RET
```

4. Frill Frontend — ML-Inspired Language for Z80

New frontend: .frl files. 881 LOC, 4 tests, 2 examples.

```
let double (x : u8) : u8 = x + x
let inc (x : u8) : u8 = x + 1
let main () : u8 = 5 |> double |> inc    -- pipe operator!
```

Compiles to: main: LD A, 11 / RET (constant folded through pipe chain).

8 frontends total: Nanz, Pascal, C89, Lizp, Lanz, PL/M, ABAP, Frill.

5. Infrastructure

- **Return-value spill tracking** — retVRegs from terminators marked as “live after” CALLs
 - **Safe tail-call optimization** — CALL→JP only when return value IS the call result
 - **dedupAsmLabels** — removes duplicate labels in hybrid LIR+PBQP output
 - **SanitizeAsmLabel** — unified @/./\$→_ sanitization + Z80 register name avoidance
 - **PBQP splice trailing data** — spill labels from PBQP available to LIR functions
-

6. E2E Corpus: 75/81 (92.6%)

By frontend

Frontend Pass Total Rate

Nanz	~17	19	~89%
ABAP	8	8	100%
Pascal	4	4	100%
C89	~44	48	~92%
Frill	2	2	100%

Remaining 6 failures

File	Root Cause	Fix needed
arena_allocator.nanz	WFC assigns SBC HL, mem (spill)	Z3 spill insertion
nc.nanz	PBQP <i>spill</i> cross-function ref	Hybrid splice refactor
assert_test.c	abs_diff: SUB before CMP	Instruction ordering

File	Root Cause	Fix needed
bench_extended.c	WFC validate: is_power_of_two	Spill insertion
import_test.c	MIR2 VM: unknown function	C89 #import
struct_promote.c	MIR2 VM: outparam bug	MIR2 codegen

7. Commit Log (24 commits)

```

07e48b33 feat(lir): Z3 SMT-based optimal register allocation
3704a653 fix(lir): addr_of symbol propagation
da545741 feat(lir): equality saturation + Z3 joint isel+regalloc
9801811d feat(lir): global store/load fusion + OpStoreGlobal/
OpLoadGlobal
02698130 fix(lir): emit string pool + fix assembly error
2f347b76 feat(lir): Z3 --z3 flag + pipeline integration
ef624b77 fix(lir): sanitize asm labels – Z80 register name
collisions
7045ec06 feat(lir): peephole – duplicate consecutive LD
elimination
c086ed21 fix(lir): cond_ret emission + assert bootstrap contract
classes
c4dcd3ce fix(lir): Z3 respects PBQP hints
53e22a0b fix(lir): 16-bit cond_ret – EX DE,HL
78f25759 fix(lir): INC/DEC pattern filter – double not increment
6f3e6381 fix(lir): fixInvalidZ80Template guard – ADD HL,HL is not
store!
068b8952 fix(lir): sanitize $ in symbols
504397e0 fix(lir): emit string pool in PBQP fallback path
f6928457 feat: Frill frontend – ML-inspired functional language
6652496c docs: Report 107
b9aa097a fix(lir): dedupAsmLabels + PBQP splice trailing data
af2ee306 fix(lir): dedupAsmLabels + trailing data fix
504397e0 fix(lir): string pool in fallback path
9d6a042b fix(lir): return-value spill infrastructure + safe tail-
call
c8993eaf feat(lir): 16-bit CMP pattern (SBC HL,DE + ADD HL,DE)

```

8. Next Steps

1. **Z3 spill insertion** — Z3 assigns to shadow/TSMC/mem, compiler inserts LD IXH,r / EXX bracket / LD (nn),A moves
2. **Equality saturation with 602K superoptimizer rules** — feed proven Z80 identities into e-graph
3. **Frill pattern matching + ADTs** — match x with None -> 0 | Some v -> v
4. **abs_diff fix** — ensure CMP runs on original values, not SUB result